Computer System Design & Application
计算机系统设计与应用A

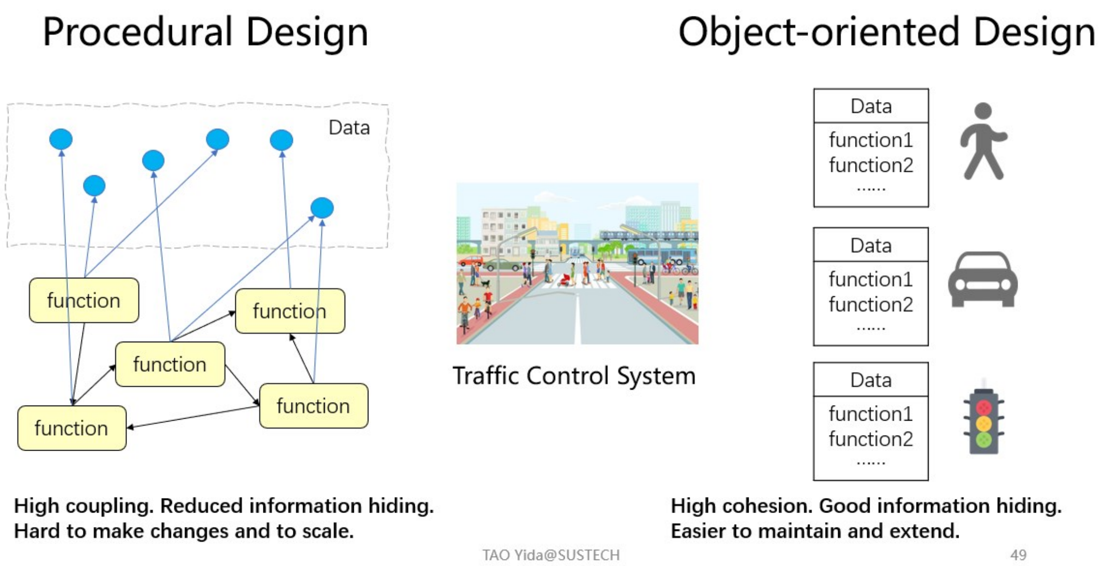
陶伊达 （TAO Yida）
taoyd@sustech.edu.cn

Lecture 3

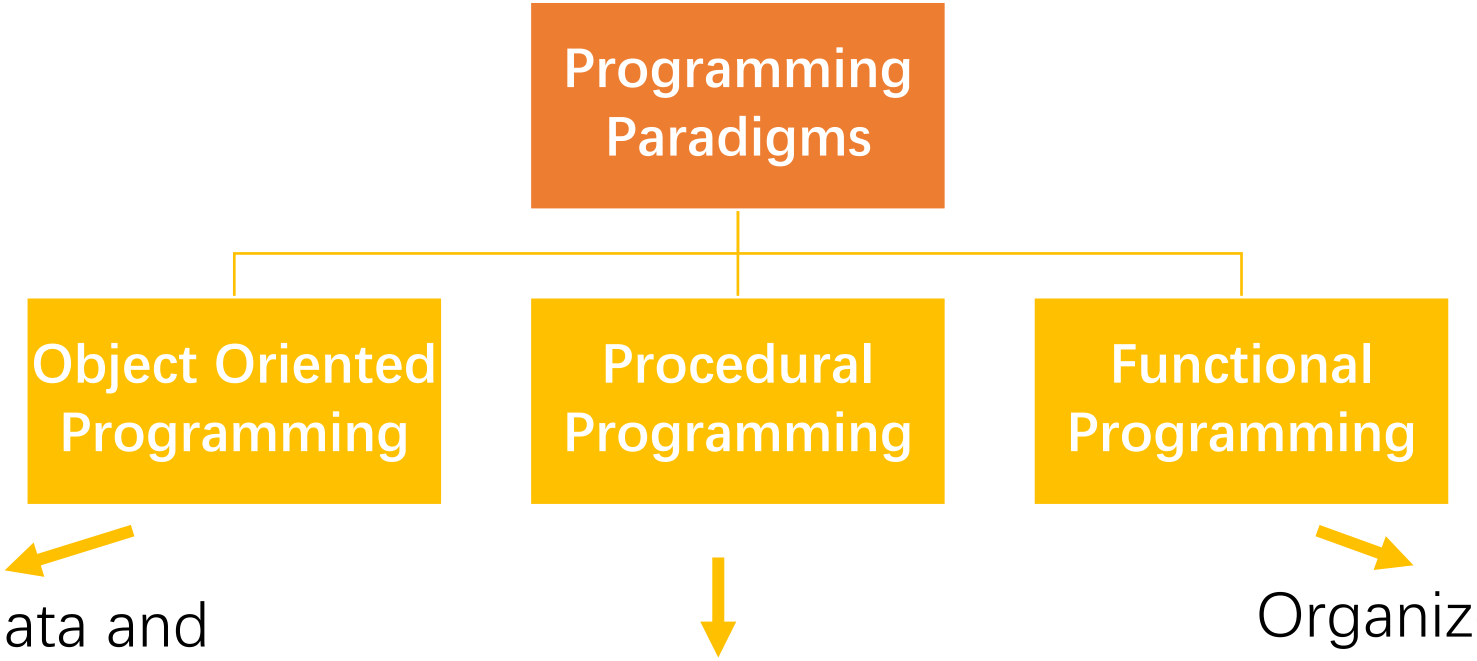
- Functional Programming
- Lambda Expressions
- Functional Interfaces

Programming Paradigm

Programming paradigm (编程范式):
the way of thinking about things and the way of solving problems



Programming Paradigms



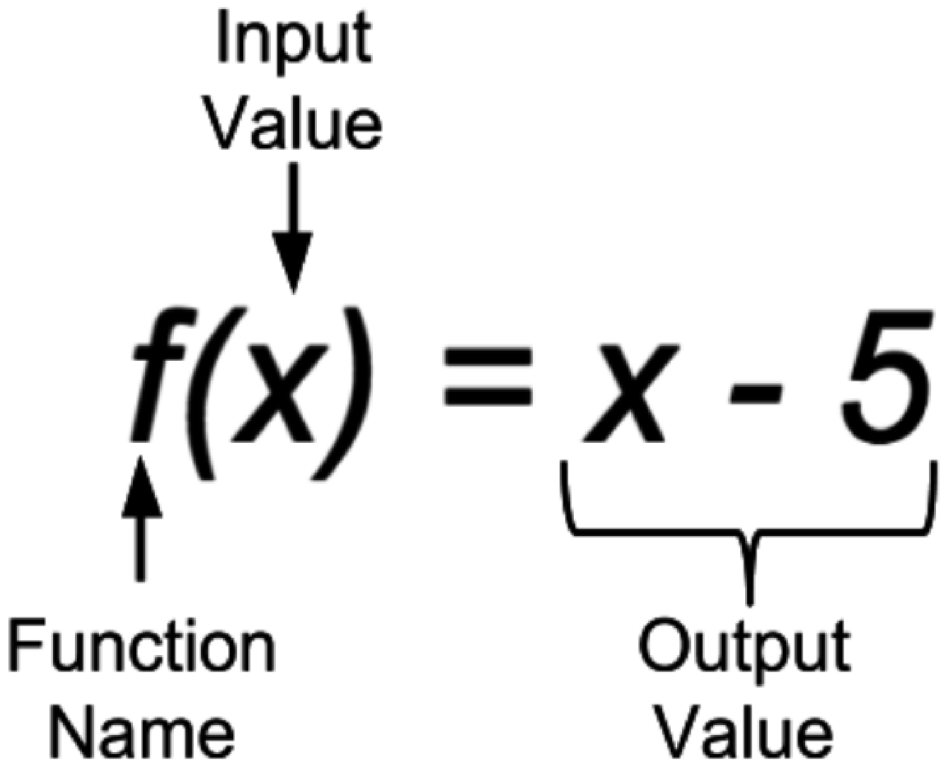
Organize data and logics in **objects**

Organize code as **procedures (过程)**

Organize code as **functions (函数)**

What is a function?

- [Mathematics] A function (函数) from a set X to a set Y assigns to each element of X exactly one element of Y
- Maps input to output



x	f(x)
1	-4
2	-3
3	-2
4	-1

Functional Programming (函数式编程)

- Basic unit of computation is function
- Primary characteristics
  - Functions are treated as first-class citizens
  - No side effects
  - Immutability
  - Recursion

First-class functions

Functions are treated like any other variables

```javascript
function me() {
  return '👨';
}

greet(me);
```

Functions can be passed as arguments

```javascript
const greet = function () {
    console.log('👋');
}

// The greet variable is now a
greet();
```

Functions can be assigned to a variable

```javascript
// #3 Return as values from other functions
function Promises() {
  return new Promise((resolve, reject) => {
    resolve('🈳');
  });
}
```

Functions can be returned

Image source : https://www.webtips.dev/webtips/javascript-interview/first-class-functions

Higher Order Functions

- A higher order function
  - Takes another function as argument
  - Returns another function as result

- A common usage scenario: data processing
  - `map()`, `filtering()`, `reduce()`, etc.
  - More on these later

Pure functions

- **Always** produce the same output for the same input
- No side effects (cannot change external states)

```
// Pure function
int add(int a, int b) {
    // always same output for same input
    return a + b;
}
```

```
// Impure function
int counter = 0;
int increment(int x) {
    // side effect: modifies global state
    counter++;
    // output changes even with same x
    return x + counter;
}
```

Pure functions

- **Always** produce the same output for the same input
- No side effects (cannot change external states)

Impure function	Side Effects
writeFile(filename)	External files are changed
updateDatabase(table)	External database table is changed
sendAjaxRequest(request)	External server state is changed

Pure functions

- **Always** produce the same output for the same input
- No side effects (cannot change external states)

Impure function	Input	Possible Output
writeFile(filename)	filename	Success or failures given file state
queryDatabase(table)	table	Different results given table state
sendAjaxRequest(request)	request	Different responses given server state

Immutability

- Variables, once defined, never change their values (predictable, safer)
- Pure functional programs do not have assignment statements (use new values instead of modifying old ones)

```
x = x + 10
```

Non-functional style

```
int plusTen(int x)
{
        return x+10;
}
```

Functional style

Recursion instead of Loops

- FP avoids explicit loops with mutable variables
- How do we perform iterations though?

$$n! = n \cdot (n-1) \cdot (n-2) \cdots 3 \cdot 2 \cdot 1$$

```
def factorial(n):
    fact = 1
    while n >= 1:
        fact = fact * n
        n = n - 1
    return fact
```

```
def factorial(n):
    if n <= 0: return 1
    return n * factorial(n-1)
```

Using recursive functions (递归)
Or higher order functions (e.g., map)

Benefits of Functional Programming

- Easy to debug, test, and parallelize
  - Same input => same output (deterministic)
  - No side effects
  - Data are immutable

- Complexity is dramatically reduced (architectural simplicity)
  - The only interaction with the external system is via the argument and return value (API)

Functional Programming Languages

- Lisp, Erlang, Haskell, Clojure, Scala, F#, Python, Javascript, Kotlin, Rust, Swift, etc.

- FP is used in big companies
  - **Whatsapp** needs only 50 engineers for its 900M users, using Erlang
  - **Huawei** adopts Rust to engineer trustworthy software systems
  - **NVIDIA** uses Haskell for the backend development of its GPUs
  - **Facebook** uses Haskell to fight spams
  - ……

Functional Programming in Java

- Different programming paradigms are not necessarily mutually exclusive
  - Python also supports OOP
  - Java also supports the functional styles of programming

- Java 8 introduces functional programming abilities
  - Lambda expressions
  - Streams API

Lecture 3

- Functional Programming
- Lambda Expressions
- Functional Interfaces

Java Lambda Expressions

- Introduced in Java 8
  - Java's first step into functional programming

- A Java lambda expression
  - is an anonymous function with no name/identifier
  - can be created without belonging to any class
  - can be passed as a parameter to another function
  - are callable anywhere in the program

Lambda Syntax

Arrow token

```
(param1, param2) -> {expressions or statements}
```

Left part – function parameters

- No function name
- Parentheses could be omitted for a single parameter
- Multiple parameters are separated by comma (,)
- `()` is used if no parameter is needed

Right part – function body

- Curly braces could be omitted for a single expression
- Multiple statements are separated by a semi-colon (;)
- Can have a `return` statement
- Local assignments and control structures (`if`, `for`) are allowed (but probably less common).

Lambda Syntax

- One statement

```
(param) -> System.out.println("1 parameter:"+ param)
```

- Multiple statements (with curly braces)

```
(first, second) ->
{
    if (first.length() < second.length()) return -1;
    else if (first.length() > second.length()) return 1;
    else return 0;
}
```

- Return

```
(param1, param2) -> {return param1 > param2;}
(param1, param2) -> param1 > param2;
```

Method Reference

A method reference is shorthand for a lambda expression that just calls one existing method. It improves readability when the lambda body is just forwarding arguments.

Lambda	Method Reference
`s -> System.out.println(s)`	`System.out::println`
`str -> Integer.parseInt(str)`	`Integer::parseInt`
`s -> s.toLowerCase()`	`String::toLowerCase`
`() -> new ArrayList<>()`	`ArrayList::new`

Lambda Usage

- Lambda: a shortcut to define an implementation of a functional interface
- Functional interface is an interface with a single abstract method (e.g., `Comparator<T>` interface only has one abstract method `int compare(T o1, T o2)`)
- We can supply a lambda expression whenever an object of a function interface is expected

Example: sorting a string list by element’s length

```
public class StringComparator implements Comparator<String>{
    public int compare(String s1, String s2) {
        return Integer.compare(s1.length(), s2.length());
    }
}
Collections.sort(strList, new StringComparator());
```

Classic way

- Explicitly creating a class that implements the Comparator interface
- Verbose

```
Collections.sort(strList, new Comparator<String>() {
        public int compare(String s1, String s2) {
            return Integer.compare(s1.length(), s2.length());
        }
});
```

Using the anonymous class

- Don’t have to declare a name for it
- Declare and instantiate the class at the same time
- Anonymous class can be used only once
- Shorter code, but still verbose

Using lambda in Java 8

```
Collections.sort(strList, (s1, s2) -> Integer.compare(s1.length(), s2.length()));
```

Matching Lambdas to Functional Interfaces

```
Collections.sort(strList, (s1, s2) -> Integer.compare(s1.length(), s2.length()));

Collections.sort(List<T> list,        Comparator<? super T> c )

                                      int compare(T o1, T o2)
```

1. Lambda is matched to the Comparator interface

2. Lambda is matched to `Comparator.compare(T o1, T o2)` method, which is the only abstract(unimplemented) method in the `Comparator` interface

3. The parameter and return type are deduced by compiler using type inference

Type Inference (类型推断)

- Compiler obtains most of the type information from *generics*
- Compiler won't be able to infer types if raw type (e.g., `List`) is used instead of the parameterized type (e.g., `List<String>`)

List is a raw type. References to generic type List<E> should be parameterized

```
List strList = new ArrayList();
strList.add("abc");
strList.add("bcd");
Collections.sort(strList, (s1, s2) -> Integer.compare(s1.length(), s2.length()));
```

The method length() is undefined for the type Object

Use `List<String> strList = new ArrayList<String>()` to remove all the warnings and errors!

Lecture 3

- Functional Programming
- Lambda Expressions
- Functional Interfaces

Built-in Functional Interfaces

- The `java.util.function` package

- Well defined set of general-purpose functional interfaces
  - All have only one abstract method
  - Lambda can be used wherever these interfaces are used
  - They are used extensively in Java class libraries, especially with the `Streams` API (later)

Consumer<T>

The `Consumer` interface has one abstract method `void accept(T t)`

represents a function that takes an argument of type T and returns nothing (consume it)

```
List<String> list = new ArrayList<>();
list.add("This");
list.add("is");
list.add("Java");
list.forEach(System.out::println);
```

```
void forEach(Consumer<? super T> action)
```

Supplier<T>

The `Supplier` interface has one abstract method `T get()`

represents a function that takes no argument and returns (supplies) a value of type T

```
Supplier<String> textSupplier = () -> "Hello!";
Supplier<Integer> integerSupplier = () -> 123;
Supplier<Double> randomSupplier = () -> Math.random();
Supplier<Double> randomSupplierMR = Math::random;

System.out.println(textSupplier.get());
System.out.println(integerSupplier.get());
System.out.println(randomSupplier.get());
System.out.println(randomSupplierMR.get());
```

Predicate<T>

The `Predicate` interface has one abstract method `boolean test(T t)` represents a function that takes a value of type T and returns a boolean

`default boolean removeIf(Predicate<? super E> filter)`

Removes all of the elements of this collection that satisfy the given predicate.

```
List<String> list = new ArrayList<>();
list.add("This");
list.add("is");
list.add("a");
list.add("Java");
list.add("Course");
list.removeIf(e -> e.length()<3);
list.forEach(System.out::println);
```

Function<T, R>

The `Function` interface has one abstract method `R apply(T t)`

represents a function that takes a value of type T and returns a value of type R

```
Function<String, Integer> strLenFunction = String::length;
String inputString = "Hello, World!";
int length = strLenFunction.apply(inputString);
```

```
Function<Integer,int[]> arrayCreator = int[]::new;
int[] intArray = arrayCreator.apply(5);
System.out.println(Arrays.toString(intArray)); // [0,0,0,0,0]
```

UnaryOperator<T>

represents a function that takes a value of type T and returns a value of type T

```
List<Integer> nums = Arrays.asList(1, 2, 3);
nums.replaceAll(x -> x * 2); // [2, 4, 6]
```

```
void replaceAll(UnaryOperator<E> operator)
```

Common Functional Interfaces

The `Operator` interfaces represent functions whose result and argument types are the same

The `Predicate` interface represents functions who take an argument and return a boolean

The `Function` interface represents functions whose result and argument types could differ

Interface	Function Signature	Example
`UnaryOperator<T>`	`T apply(T t)`	`String::toLowerCase`
`BinaryOperator<T>`	`T apply(T t1, T t2)`	`BigInteger::add`
`Predicate<T>`	`boolean test(T t)`	`Collection::isEmpty`
`Function<T,R>`	`R apply(T t)`	`Arrays::asList`
`Supplier<T>`	`T get()`	`Instant::now`
`Consumer<T>`	`void accept(T t)`	`System.out::println`

Table from "Effective Java"

More Use Cases

https://docs.oracle.com/javase/tutorial/java/javaOO/lambdaexpressions.html#approach5

Lambda Expressions

One issue with anonymous classes is that if the implementation of your anonymous class is very simple, su
unclear. In these cases, you're usually trying to pass functionality as an argument to another method, such
functionality as method argument, or code as data.

The previous section, Anonymous Classes, shows you how to implement a base class without giving it a na
class seems a bit excessive and cumbersome. Lambda expressions let you express instances of single-me

This section covers the following topics:

- Ideal Use Case for Lambda Expressions
  - Approach 1: Create Methods That Search for Members That Match One Characteristic
  - Approach 2: Create More Generalized Search Methods
  - Approach 3: Specify Search Criteria Code in a Local Class
  - Approach 4: Specify Search Criteria Code in an Anonymous Class
  - Approach 5: Specify Search Criteria Code with a Lambda Expression
  - Approach 6: Use Standard Functional Interfaces with Lambda Expressions
  - Approach 7: Use Lambda Expressions Throughout Your Application
  - Approach 8: Use Generics More Extensively
  - Approach 9: Use Aggregate Operations That Accept Lambda Expressions as Parameters

Next Lecture

- Java 8 Stream API